

# MODULE 1 — INTRODUCTION TO JAVA

## 1. Basics of Java Programming

### 1.1 What is Java?

Java is a **high-level, object-oriented programming language** used to develop different types of applications. It allows programmers to write instructions in a simple, human-readable form instead of directly working with machine language. Java is widely used for enterprise applications, web applications, desktop software, and many other systems.

#### *Real-Time Example*

A **college management system** can use Java to manage student details, attendance, marks, fees, and examination information.

#### *Simple Example*

```
public class Demo {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java");  
    }  
}
```

#### *Output*

```
Welcome to Java
```

### 1.2 Why is Java Called a High-Level Language?

Java is called a high-level language because its syntax is **easy for humans to read and understand**. Programmers can write instructions using English-like words without knowing the internal CPU instructions or memory operations. This makes Java easier to develop, debug, and maintain.

#### *Example*

```
int age = 20;
```

A programmer can easily understand that the value 20 is being stored in age.

Compare this with machine language:

```
10110010 11001001
```

The machine can understand it, but humans cannot easily understand it.

### 1.3 Why is Java Platform Independent?

Java is platform independent because a Java program is not directly converted into machine code for one particular operating system. The Java compiler converts the program into **bytecode**, which can run on different operating

systems using their respective JVM.

### Example

The same Java program can run on:

```
Windows  
Linux  
macOS
```

without rewriting the program.

### Remember

**Write Once, Run Anywhere (WORA)**

## 1.4 What is JVM?

**JVM stands for Java Virtual Machine.** It is the component that executes Java bytecode and converts it into instructions that the particular computer can understand. JVM is different for different operating systems, which helps Java achieve platform independence.

### Execution

```
Java Program  
↓  
Compiler  
↓  
Bytecode  
↓  
JVM  
↓  
Output
```

## 1.5 What is JDK?

**JDK stands for Java Development Kit.** It contains the tools required to develop, compile, debug, and run Java programs. A programmer normally installs the JDK when developing Java applications.

### Example

The Java compiler:

```
javac
```

is one of the important tools provided by the JDK.

## 1.6 What is JRE?

**JRE stands for Java Runtime Environment.** It provides the environment required to run Java applications and contains the JVM along with required Java libraries. A user who only needs to run Java applications does not need all the development tools provided by the JDK.

### Relationship

JDK  
↓  
JRE  
↓  
JVM

## 1.7 First Java Program

A Java program normally contains a class and a `main()` method from where execution begins. The `System.out.println()` statement is used to display information on the screen.

```
public class FirstProgram {  
  
    public static void main(String[] args) {  
  
        System.out.println("Welcome to Java");  
  
    }  
}
```

### Output

Welcome to Java

### Important Parts

```
public class FirstProgram
```

Creates a class named `FirstProgram`.

```
public static void main(String[] args)
```

Starting point of the Java program.

```
System.out.println()
```

Displays output on the screen.

## 2. Data Types

### 2.1 What is a Data Type?

A data type tells the Java compiler **what kind of value a variable will store**. Different data require different data types, such as integers for age, decimal values for percentage, characters for grades, and Boolean values for true/false conditions.

### Real-Time Example

Student information can contain:

```
Age      → 20
Percentage → 85.5
Grade    → A
Passed   → true
```

Therefore, different data types are required.

## 2.2 Primitive Data Types

Primitive data types are the **basic built-in data types provided by Java**. They are used to store simple values such as numbers, characters, and true/false values. Java provides eight primitive data types.

| Data Type | Purpose               | Example |
|-----------|-----------------------|---------|
| byte      | Small integer         | 10      |
| short     | Integer               | 1000    |
| int       | Normal integer        | 50000   |
| long      | Large integer         | 100000L |
| float     | Decimal               | 85.5f   |
| double    | Large/precise decimal | 85.567  |
| char      | Single character      | 'A'     |
| boolean   | True/False            | true    |

### 2.3 int

The `int` data type is used to store **whole numbers without decimal points**. It is one of the most commonly used data types when working with numbers such as age, marks, student count, or quantity.

```
int age = 20;
int marks = 85;
int students = 50;
```

### 2.4 float

The `float` data type is used to store **decimal numbers** where normal precision is sufficient. When assigning a decimal value to a float variable, we normally add `f` at the end of the value.

```
float percentage = 85.5f;
float price = 99.50f;
```

### 2.5 double

The `double` data type is used for decimal values requiring **more precision than float**. It is commonly used for calculations involving measurements, scientific values, prices, and percentages.

```
double salary = 35678.75;
double distance = 125.678;
```

## 2.6 char

The `char` data type is used to store **a single character**. A character must be written inside single quotation marks.

```
char grade = 'A';
char section = 'B';
```

■ **Wrong:**

```
char grade = "A";
```

## 2.7 boolean

The `boolean` data type is used when a program needs only **two possible values: true or false**. It is mainly used in decision-making and conditions.

```
boolean passed = true;
boolean loggedIn = false;
```

### *Real-Time Example*

```
Student passed? → true
Student passed? → false
```

## 3. Variables

### 3.1 What is a Variable?

A variable is a **named memory location used to store a value**. The value stored in a variable can be used later in the program and, in many cases, can be changed during execution.

### *Real-Time Example*

Think of a variable as a labelled box:

The diagram shows a rectangular box with the label 'age' at the top left. Inside the box, the number '20' is centered. The box is composed of a grid of small squares, with the label and the value '20' occupying specific positions within this grid.

Java:

```
int age = 20;
```

## 3.2 Variable Declaration

Variable declaration means telling Java **the variable's name and the type of data it will store**. At this stage, we are creating the variable but have not necessarily assigned a value to it.

```
int age;
```

Here:

```
int → Data type  
age → Variable name
```

## 3.3 Variable Initialization

Initialization means **assigning an initial value to a variable**. We can declare a variable first and assign the value later, or we can do both in one statement.

```
int age;  
age = 20;
```

Or:

```
int age = 20;
```

## 3.4 Local Variable

A local variable is a variable declared **inside a method or block**. It can be accessed only within that particular method or block where it is declared.

```
public static void main(String[] args) {  
  
    int age = 20;  
  
    System.out.println(age);  
}
```

Here age is a local variable.

## 3.5 Instance Variable

An instance variable is declared **inside a class but outside methods**. Each object created from the class can have its own copy of an instance variable.

```
class Student {  
  
    int age;  
  
}
```

Here age is an instance variable.

## 3.6 Static Variable

A static variable belongs to the **class rather than individual objects**. Because it is shared, all objects of that class can access the same static variable.

```
class Student {  
  
    static String college = "KLE";  
  
}
```

## 4. Operators

### 4.1 What is an Operator?

An operator is a **special symbol used to perform an operation on values or variables**. Java provides different operators for calculations, comparisons, logical decisions, assigning values, and other operations.

#### Example

```
int a = 10;  
int b = 20;  
  
int sum = a + b;
```

Here:

```
a, b → operands  
+    → operator
```

### 4.2 Arithmetic Operators

Arithmetic operators are used to perform **basic mathematical calculations** such as addition, subtraction, multiplication, and division. The % operator is especially useful because it gives the remainder after division.

```
int a = 20;  
int b = 6;  
  
System.out.println(a + b);  
System.out.println(a - b);  
System.out.println(a * b);  
System.out.println(a / b);  
System.out.println(a % b);
```

Output:

```
26  
14  
120  
3  
2
```

## 4.3 Relational Operators

Relational operators are used to **compare two values**. The result of a comparison is always either `true` or `false`, which makes these operators very useful in decision-making.

```
int a = 10;
int b = 20;

System.out.println(a > b);
System.out.println(a < b);
System.out.println(a == b);
```

Output:

```
false
true
false
```

## 4.4 Logical Operators

Logical operators are used to **combine or reverse conditions**. `&&` means AND, `||` means OR, and `!` means NOT; they are commonly used with `if` statements.

```
int age = 20;

System.out.println(age >= 18 && age <= 60);
```

Output:

```
true
```

### *Real-Time Example*

A student can attend an exam if:

```
Attendance >= 75 AND Hall Ticket = available
```

## 4.5 Assignment Operators

Assignment operators are used to **store or update values in variables**. The basic assignment operator is `=`, while operators such as `+=`, `-=`, `*=`, and `/=` provide shorter ways to update a variable.

```
int a = 10;

a += 2;
```

This is the same as:

```
a = a + 2;
```



Therefore:

```
a = 12
```

## 4.6 Unary Operators

Unary operators work with **only one operand**. They are commonly used to increase or decrease a value by one, change the sign of a value, or reverse a Boolean condition.

Important unary operators:

```
++  
--  
+  
-  
!
```

Example:

```
int a = 10;  
  
a++;  
  
System.out.println(a);
```

Output:

```
11
```

## 4.7 Ternary Operator

The ternary operator is a **short form of a simple if-else statement**. It checks a condition and selects one of two values depending on whether the condition is true or false.

```
int marks = 60;  
  
String result = (marks >= 35) ? "Pass" : "Fail";  
  
System.out.println(result);
```

Output:

```
Pass
```

# 5. CONTROL STRUCTURES

## 5.1 What is a Control Structure?

A control structure determines **the order in which statements are executed in a program**. It allows a program to make decisions, repeat statements, or change the normal flow of execution. Without control structures, a program would simply execute every statement from top to bottom.

## Real-Time Example

In a college result system:

```
Marks >= 35 → Pass  
Marks < 35 → Fail
```

The program must **make a decision** based on the marks. This is done using a control structure.

## Types of Control Structures

Java control structures can be broadly divided into:

1. **Selection / Decision Making**
2. **Looping / Iteration**
3. **Branching / Jump Statements**

## 5.2 Selection Statements

Selection statements are used when a program needs to **choose between different actions based on a condition**. The program checks the condition and executes the appropriate block of code. Java provides `if`, `if-else`, `else-if`, and `switch` for selection.

## 5.3 if Statement

The `if` statement executes a block of code **only when the given condition is true**. If the condition is false, the block is simply skipped and the program continues with the next statement.

### Syntax

```
if (condition) {  
    // statements  
}
```

### Example

```
int marks = 70;  
  
if (marks >= 35) {  
    System.out.println("Pass");  
}
```

### Output

```
Pass
```

## Real-Time Example

If the temperature is greater than 30°C, display "Hot Day".

```
int temperature = 35;

if (temperature > 30) {
    System.out.println("Hot Day");
}
```

## 5.4 if-else Statement

The `if-else` statement is used when there are **two possible outcomes**. If the condition is true, the `if` block executes; otherwise, the `else` block executes.

### Syntax

```
if (condition) {
    // true block
} else {
    // false block
}
```

### Example

```
int marks = 30;

if (marks >= 35) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

### Output

```
Fail
```

### Real-Time Example

A login system can check:

```
Correct password → Login Successful
Wrong password  → Invalid Password
```

## 5.5 else-if Ladder

An `else-if` ladder is used when we need to check **more than two conditions**. The conditions are checked from top to bottom, and when one condition becomes true, its block is executed and the remaining conditions are skipped.

### Example: Student Grade

```

int marks = 82;

if (marks >= 90) {
    System.out.println("Grade A+");
}
else if (marks >= 80) {
    System.out.println("Grade A");
}
else if (marks >= 70) {
    System.out.println("Grade B");
}
else if (marks >= 60) {
    System.out.println("Grade C");
}
else {
    System.out.println("Fail");
}

```

### Output

```
Grade A
```

### Real-Time Example

This can be used in a **college grading system** to assign grades according to marks.

## 5.6 Nested if

A nested **if** means placing **one if statement inside another if statement**. It is useful when one decision needs to be checked only after another condition has already been satisfied.

### Real-Time Example

A student can write an examination only when:

1. Attendance is sufficient.
2. Hall ticket is available.

```

int attendance = 80;
boolean hallTicket = true;

if (attendance >= 75) {

    if (hallTicket) {
        System.out.println("Allowed for Exam");
    }

}

```

### Output

```
Allowed for Exam
```

## 5.7 switch Statement

The **switch** statement is used when we want to select **one option from several fixed choices**. It compares an expression with different **case** values and executes the matching case.

### Example

```
int day = 2;

switch (day) {

    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;

    default:
        System.out.println("Invalid Day");
}
```

### Output

```
Tuesday
```

### Real-Time Example

A menu-driven application can have:

```
1 → Add Student
2 → Delete Student
3 → Search Student
4 → Exit
```

switch can be used to select the required operation.

## 5.8 Why is break Used in switch?

The break statement stops the execution of the switch after the matching case is completed. Without break, Java may continue executing the statements in the following cases.

```
case 1:
    System.out.println("Monday");
    break;
```

**Remember:** break means stop here.

## 6. LOOPING STATEMENTS

### 6.1 What is a Loop?

A loop is used to **execute the same block of code repeatedly** while a particular condition is satisfied. Loops save time and reduce the need to write the same statements again and again.

### Real-Time Example

Suppose a teacher wants to display:

```
Welcome BCA Students
```

100 times.

Instead of writing `System.out.println()` 100 times, we can use a loop.

## Types of Loops

Java provides three main loops:

1. for
2. while
3. do-while

### 6.2 for Loop

The `for` loop is generally used when we **know how many times a task needs to be repeated**. It contains initialization, condition, and increment/decrement in a single line, making it convenient for counting operations.

#### Syntax

```
for (initialization; condition; increment/decrement) {  
    // statements  
}
```

#### Example

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

#### Output

```
1  
2  
3  
4  
5
```

#### How it works

```
i = 1  
↓  
Check i <= 5  
↓  
Print i  
↓  
i++  
↓  
Check again
```

## 6.3 while Loop

The `while` loop executes a block of code **as long as its condition remains true**. The condition is checked before the loop body executes, so it is possible for the loop body to execute zero times.

### Example

```
int i = 1;

while (i <= 5) {

    System.out.println(i);

    i++;

}
```

### Output

```
1
2
3
4
5
```

### Real-Time Example

A login system can keep asking for a password **while the password is incorrect**.

## 6.4 do-while Loop

The `do-while` loop executes its body **at least once before checking the condition**. This makes it useful when the operation must happen at least one time, such as displaying a menu.

### Syntax

```
do {
    // statements
} while (condition);
```

### Example

```
int i = 1;

do {

    System.out.println(i);

    i++;

} while (i <= 5);
```

### Output

```
1
2
3
4
5
```

## Difference Between `while` and `do-while`

Suppose:

```
int i = 10;
```

`while`

```
while (i < 5) {  
    System.out.println(i);  
}
```

Output:

```
No output
```

`do-while`

```
do {  
    System.out.println(i);  
} while (i < 5);
```

Output:

```
10
```

### Important Point

`while` → condition checked **first**

`do-while` → condition checked **after execution**

Therefore:

**do-while executes at least once.**

## 6.5 `break` Statement

The `break` statement is used to **immediately terminate a loop** when a particular condition occurs. After `break` executes, control moves to the statement following the loop.

### Example

```
for (int i = 1; i <= 5; i++) {  
  
    if (i == 3) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

**Output**



```
1  
2
```

When `i` becomes 3, the loop stops.

## 6.6 continue Statement

The `continue` statement is used to **skip the current iteration** and move to the next iteration of the loop. Unlike `break`, it does not terminate the entire loop.

### Example

```
for (int i = 1; i <= 5; i++) {  
  
    if (i == 3) {  
        continue;  
    }  
  
    System.out.println(i);  
}
```

### Output

```
1  
2  
4  
5
```

Here, only 3 is skipped.

### Easy way to remember

```
break    → Stop the loop  
continue → Skip this iteration
```

## 7. JAVA METHODS

### 7.1 What is a Method?

A method is a **named block of code designed to perform a particular task**. Instead of writing the same code repeatedly, we can place it inside a method and call it whenever required.

### Real-Time Example

A calculator performs different operations such as:

```
add()  
subtract()  
multiply()  
divide()
```

Each operation can be written as a separate method.

## 7.2 Why Do We Use Methods?

Methods help us divide a large program into **small, manageable parts**. They reduce repeated code, improve readability, and make programs easier to test, debug, and maintain.

## 7.3 Simple Method

```
public class Demo {  
  
    static void greet() {  
  
        System.out.println("Welcome to Java");  
  
    }  
  
    public static void main(String[] args) {  
  
        greet();  
  
    }  
  
}
```

### Output

```
Welcome to Java
```

Here:

```
greet();
```

is the **method call**.

## 7.4 Method with Parameters

A method can accept values from the calling program through **parameters**. Parameters allow the same method to work with different input values.

### Example

```
public class Demo {  
  
    static void add(int a, int b) {  
  
        System.out.println("Sum = " + (a + b));  
  
    }  
  
    public static void main(String[] args) {  
  
        add(10, 20);  
  
    }  
  
}
```

### Output

```
Sum = 30
```

Here:

```
a, b      → Parameters  
10, 20    → Arguments
```

## 7.5 Method Returning a Value

A method can calculate a result and **send that result back to the calling statement using** `return`. The return type of the method specifies what type of value it will return.

### Example

```
public class Demo {  
  
    static int square(int n) {  
  
        return n * n;  
  
    }  
  
    public static void main(String[] args) {  
  
        int result = square(5);  
  
        System.out.println(result);  
  
    }  
}
```

### Output

```
25
```

## 7.6 void

`void` means that the method **does not return a value**.

```
static void display() {  
  
    System.out.println("Hello");  
  
}
```

## 7.7 Four Basic Types of Methods

| Type                    | Example                             |
|-------------------------|-------------------------------------|
| No parameter, no return | <code>void display()</code>         |
| Parameter, no return    | <code>void add(int a, int b)</code> |

| Type                 | Example                            |
|----------------------|------------------------------------|
| No parameter, return | <code>int getNumber()</code>       |
| Parameter, return    | <code>int add(int a, int b)</code> |

## 8. METHOD OVERLOADING

### 8.1 What is Method Overloading?

Method overloading means defining **multiple methods with the same name but different parameter lists** in the same class. It allows us to perform similar operations using one meaningful method name.

#### *Real-Time Example*

A calculator can have:

```
add(10, 20)
add(10, 20, 30)
```

The operation is the same—**addition**—but the number of inputs is different.

### 8.2 Example

```
public class Addition {

    static int add(int a, int b) {
        return a + b;
    }

    static int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {

        System.out.println(add(10, 20));

        System.out.println(add(10, 20, 30));

    }
}
```

#### *Output*

```
30
60
```

### 8.3 Ways to Overload a Method

Methods can be overloaded by changing:

#### *Number of parameters*

```
add(int, int)
add(int, int, int)
```

### Data type

```
display(int)
display(double)
```

### Order of parameters

```
show(int, String)
show(String, int)
```

### Important

Changing **only the return type** does not create method overloading.

## 9. MATH CLASS

### 9.1 What is Math Class?

`Math` is a predefined Java class that provides methods for performing **common mathematical operations**. Instead of writing mathematical calculations manually, programmers can use these ready-made methods.

#### Real-Time Example

A banking or billing application may need to calculate:

- Maximum value
- Minimum value
- Square root
- Power
- Rounding

The `Math` class provides methods for these operations.

### 9.2 Important Math Methods

| Method                    | Purpose                 |
|---------------------------|-------------------------|
| <code>Math.sqrt()</code>  | Square root             |
| <code>Math.pow()</code>   | Power                   |
| <code>Math.max()</code>   | Larger value            |
| <code>Math.min()</code>   | Smaller value           |
| <code>Math.abs()</code>   | Positive/absolute value |
| <code>Math.ceil()</code>  | Round upward            |
| <code>Math.floor()</code> | Round downward          |

| Method                     | Purpose                  |
|----------------------------|--------------------------|
| <code>Math.round()</code>  | Round to nearest integer |
| <code>Math.random()</code> | Generate random value    |

### 9.3 `Math.sqrt()`

Returns the **square root** of a number.

```
System.out.println(Math.sqrt(25));
```

Output:

```
5.0
```

### 9.4 `Math.pow()`

Returns the value of one number raised to a power.

```
System.out.println(Math.pow(2, 3));
```

Output:

```
8.0
```

Because:

```
23 = 2 × 2 × 2 = 8
```

### 9.5 `Math.max()` and `Math.min()`

`Math.max()` returns the larger of two values, while `Math.min()` returns the smaller value.

```
System.out.println(Math.max(10, 20));
System.out.println(Math.min(10, 20));
```

Output:

```
20
10
```

### 9.6 `Math.abs()`

`Math.abs()` returns the **absolute value**, meaning it removes the negative sign from a number.

```
System.out.println(Math.abs(-25));
```

Output:

```
25
```

## 9.7 Math.ceil()

`Math.ceil()` rounds a decimal value **upward to the next integer value**.

```
System.out.println(Math.ceil(5.2));
```

Output:

```
6.0
```

## 9.8 Math.floor()

`Math.floor()` rounds a decimal value **downward to the lower integer value**.

```
System.out.println(Math.floor(5.9));
```

Output:

```
5.0
```

## 9.9 Math.round()

`Math.round()` rounds a decimal number to the **nearest whole number**.

```
System.out.println(Math.round(5.6));
```

Output:

```
6
```

## 9.10 Math.random()

`Math.random()` generates a random decimal number from **0.0 up to but not including 1.0**. It can be useful for games, simulations, random selections, and generating random numbers.

```
System.out.println(Math.random());
```

Possible output:

```
0.738421
```

### Random number from 1 to 100

```
int number = (int)(Math.random() * 100) + 1;

System.out.println(number);
```

Possible output:

```
57
```

## 10. ARRAYS IN JAVA

### 10.1 What is an Array?

An array is a collection of **multiple values of the same data type stored under one variable name**. It is useful when we need to store a large number of related values, such as marks of students or prices of products.

#### Real-Time Example

Instead of:

```
int mark1 = 80;
int mark2 = 75;
int mark3 = 90;
int mark4 = 65;
```

we can use:

```
int[] marks = {80, 75, 90, 65};
```

Now all marks are stored using one variable called `marks`.

### 10.2 Array Index

Each element of an array has a position called an **index**. In Java, array indexing always starts from **0**, so the first element is at index 0 and the last element is at `length - 1`.

|        |    |    |    |    |
|--------|----|----|----|----|
| Value: | 80 | 75 | 90 | 65 |
| Index: | 0  | 1  | 2  | 3  |

Therefore:

```
marks[0] → 80
marks[1] → 75
marks[2] → 90
marks[3] → 65
```

### 10.3 Creating an Array



We can create and initialize an array in a single statement.

```
int[] marks = {80, 75, 90, 65, 85};
```

Java automatically creates space for five integer values.

## 10.4 Creating an Array Using `new`

We can also specify the size first and assign values later.

```
int[] marks = new int[5];

marks[0] = 80;
marks[1] = 75;
marks[2] = 90;
marks[3] = 65;
marks[4] = 85;
```

Here, `new int[5]` creates an array capable of storing **5 integers**.

## 10.5 Display Array Elements

A loop is normally used to access all elements of an array one by one. The loop starts at index 0 and continues until the last valid index.

```
public class ArrayDemo {

    public static void main(String[] args) {

        int[] marks = {80, 75, 90, 65, 85};

        for (int i = 0; i < marks.length; i++) {

            System.out.println(marks[i]);

        }

    }

}
```

### *Output*

```
80
75
90
65
85
```

## 10.6 What is `length`?

The `length` property tells us the **number of elements present in an array**. It is useful in loops because we do not need to manually specify the size of the array.

```
int[] marks = {80, 75, 90, 65, 85};  
  
System.out.println(marks.length);
```

Output:

```
5
```

## 10.7 Sum of Array Elements

We can use a loop to visit every element and continuously add it to a `sum` variable. This is a common beginner program and helps students understand how arrays and loops work together.

```
public class ArraySum {  
  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 30, 40, 50};  
  
        int sum = 0;  
  
        for (int i = 0; i < numbers.length; i++) {  
  
            sum = sum + numbers[i];  
  
        }  
  
        System.out.println("Sum = " + sum);  
    }  
}
```

**Output**

```
Sum = 150
```

## 10.8 Find Largest Element

To find the largest element, we initially assume that the first element is the largest. We then compare every remaining element with it and update the `largest` variable whenever we find a bigger value.

```
public class Largest {  
  
    public static void main(String[] args) {  
  
        int[] numbers = {25, 10, 45, 30, 15};  
  
        int largest = numbers[0];  
  
        for (int i = 1; i < numbers.length; i++) {  
  
            if (numbers[i] > largest) {  
                largest = numbers[i];  
            }  
  
        }  
  
        System.out.println("Largest = " + largest);  
    }  
}
```

### Output

```
Largest = 45
```

## 10.9 Two-Dimensional Array

A two-dimensional array stores data in the form of **rows and columns**, similar to a table. It is commonly used for matrices, marks tables, seating arrangements, and other tabular data.

### Example

```
int[][] numbers = {  
    {10, 20, 30},  
    {40, 50, 60}  
};
```

It represents:

```
10  20  30  
40  50  60
```

Accessing:

```
System.out.println(numbers[1][2]);
```

Output:

```
60
```

## ■ MODULE 1 — FINAL REVISION

Students should remember these core ideas:

| Topic       | What students should understand                    |
|-------------|----------------------------------------------------|
| Java        | A high-level, object-oriented programming language |
| JVM         | Executes Java bytecode                             |
| JDK         | Used to develop Java programs                      |
| JRE         | Provides environment to run Java programs          |
| Data Type   | Defines what type of value can be stored           |
| Variable    | Named storage for a value                          |
| Operator    | Performs an operation                              |
| if          | Makes a decision                                   |
| switch      | Selects one option from multiple choices           |
| Loop        | Repeats statements                                 |
| for         | Suitable when repetitions are generally known      |
| while       | Checks condition before execution                  |
| do-while    | Executes at least once                             |
| Method      | Reusable block of code                             |
| Overloading | Same method name with different parameters         |
| Math Class  | Provides mathematical functions                    |
| Array       | Stores multiple values of the same type            |

# MODULE 1 — IMPORTANT QUESTIONS

## 2 MARKS

1. What is Java? Mention any two features of Java.
2. What is bytecode?
3. Expand JDK, JRE and JVM.
4. List the primitive data types in Java.
5. What is a variable? Give an example.
6. What is type casting?
7. What are operators? Mention their types.
8. What is the difference between = and ==?
9. What is a control statement?
10. What is a method?
11. What is method overloading?
12. What is the Math class? Mention any four Math methods.
13. What is an array?
14. How do you declare and initialize an array?
15. What is the difference between `break` and `continue`?

## 5 MARKS

1. Explain the primitive data types in Java with suitable examples.
2. Explain variable declaration, initialization and type casting in Java with examples.
3. Explain different types of operators in Java with suitable examples.
4. Explain the `Math` class and its commonly used methods with examples.
5. Explain one-dimensional arrays in Java with declaration and initialization.
6. Explain two-dimensional arrays in Java with a suitable example.

## 10 MARKS

1. Explain the features of Java in detail.
2. Explain the basic structure of a Java program and describe its components with a suitable example.
3. Explain selection statements in Java with syntax and suitable examples.
4. Explain looping statements in Java with syntax and suitable examples.
5. Explain methods in Java and method overloading with suitable examples.
6. Explain arrays in Java and discuss one-dimensional and two-dimensional arrays with suitable programs.